

將 ASP.NET Core 應用程式部署至 Google Kubernetes Engine，並使用 Istio (第 1 部分)

來源：<https://codelabs.developers.google.com/codelabs/cloud-istio-aspnetcore-part1?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將瞭解如何運用 Istio 將 ASP.NET Core 應用程式部署至 Google Kubernetes Engine。

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [在 Cloud Shell 中建立 ASP.NET Core 應用程式](#)
4. [執行 ASP.NET Core 應用程式](#)
5. [將 ASP.NET Core 應用程式封裝至 Docker 容器](#)
6. [使用 Istio 建立 Kubernetes/GKE 叢集](#)
7. [驗證安裝](#)
8. [部署應用程式](#)
9. [測試應用程式](#)
10. [恭喜！](#)
11. [清除所用資源](#)

1. 總覽

[ASP.NET Core](#) 是開放原始碼的跨平台架構，可使用 C# 程式設計語言建構現代化雲端應用程式和連網應用程式。

[Kubernetes](#) 是開放原始碼系統，可自動部署、調度資源及管理容器化應用程式。[Istio](#) 是開放式架構，可用於連結、保護、管理及監控服務。

在本實驗室的第一部分，您會將簡單的 ASP.NET Core 應用程式部署至 [Google Kubernetes Engine \(GKE\)](#) 上執行的 Kubernetes，並設定由 Istio 管理。

在本實驗室的[第二部分](#)，您將進一步探索 Istio 的功能，例如指標、追蹤、動態流量管理、錯誤注入等。

課程內容

- 瞭解如何在 Docker 容器中建立及封裝簡易的 ASP.NET Core 應用程式。
- 如何透過 Google Kubernetes Engine (GKE) 建立 Kubernetes 叢集。
- 如何在 GKE 的 Kubernetes 叢集上安裝 Istio。
- 如何部署 ASP.NET Core 應用程式，並設定由 Istio 管理應用程式流量。

軟硬體需求

- [Chrome](#) 或 [Firefox](#) 瀏覽器
-

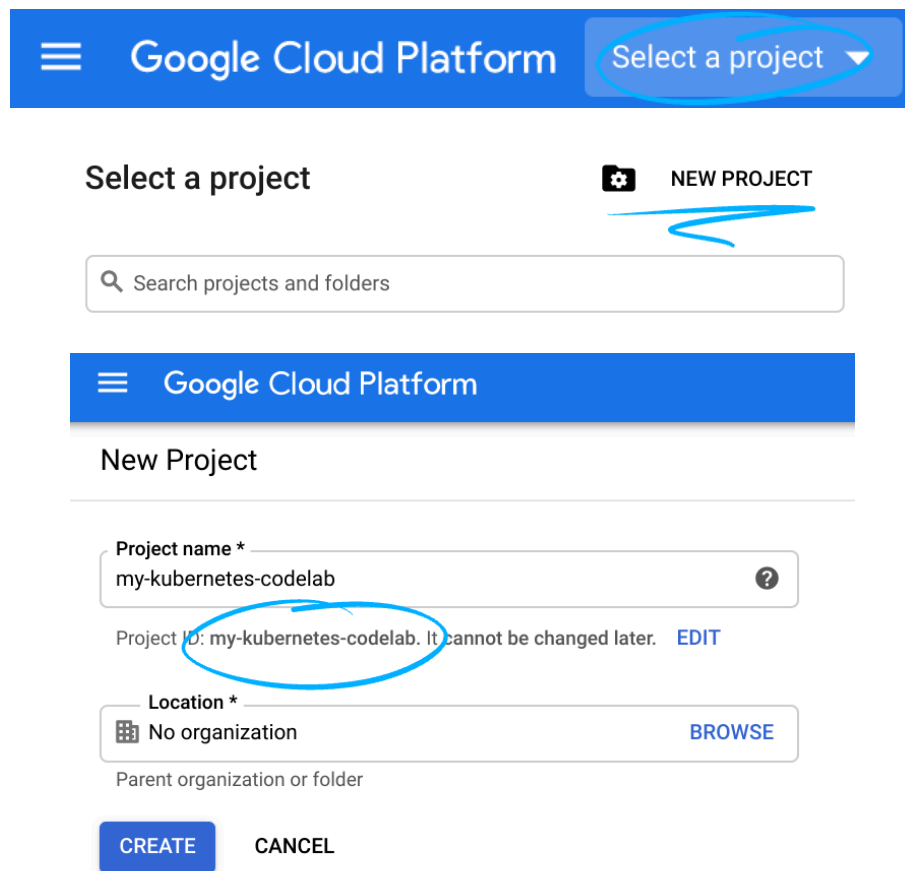
步驟 2 · 約 5 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：`console.cloud.google.com`。



The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown menu. Below this, there's a 'Select a project' section with a search bar and a 'NEW PROJECT' button. The 'NEW PROJECT' button is circled in blue. Below the search bar, there's a 'New Project' form. The 'Project name' field is filled with 'my-kubernetes-codelab' and has a question mark icon. The 'Project ID' field is filled with 'my-kubernetes-codelab' and is circled in blue. Below the 'Project ID' field, there's a note: 'Project ID: my-kubernetes-codelab. It cannot be changed later. EDIT'. The 'Location' field is filled with 'No organization' and has a 'BROWSE' button. At the bottom, there are 'CREATE' and 'CANCEL' buttons. The 'CREATE' button is highlighted.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然您可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您會使用 [Google Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。

啟用 Cloud Shell

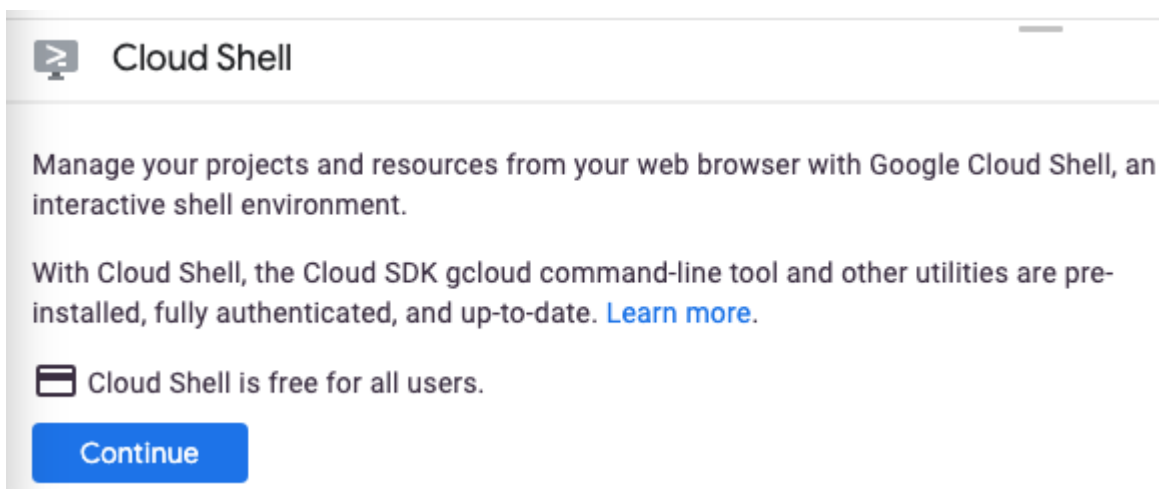
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



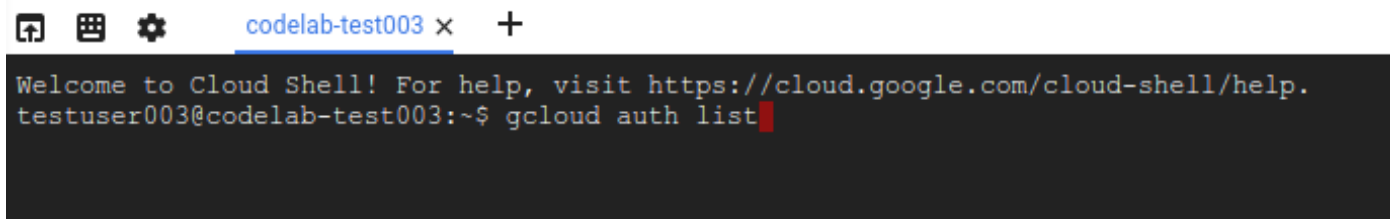
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於摺疊式選單下方)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

步驟 3 · 約 2 分鐘

3. 在 Cloud Shell 中建立 ASP.NET Core 應用程式

在 Cloud Shell 提示中，您可以檢查 dotnet 指令列工具的版本，確認工具是否已安裝。這會列印已安裝的 dotnet 指令列工具版本：

```
dotnet --version
```

接著，建立新的 ASP.NET Core 網頁應用程式基本架構。

```
dotnet new mvc -o HelloWorldAspNetCore
```

這個指令會建立專案並還原依附元件，顯示的訊息應如下所示。

```
Restore completed in 11.44 sec for HelloWorldAspNetCore.csproj.
```

```
Restore succeeded.
```

步驟 4 · 約 3 分鐘

4. 執行 ASP.NET Core 應用程式

我們即將執行應用程式。請前往應用程式資料夾。

```
cd HelloWorldAspNetCore
```

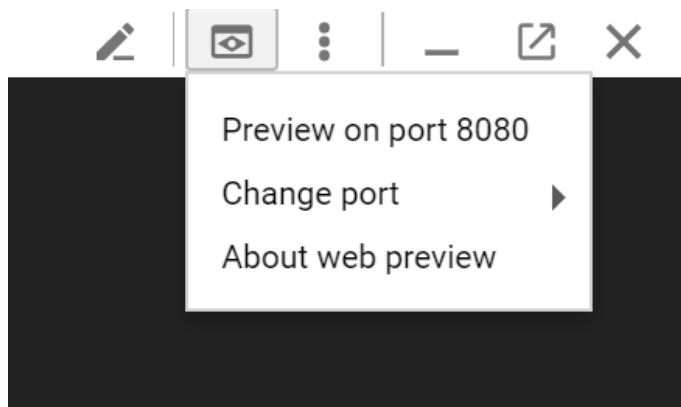
最後，執行應用程式。

```
dotnet run --urls=http://localhost:8080
```

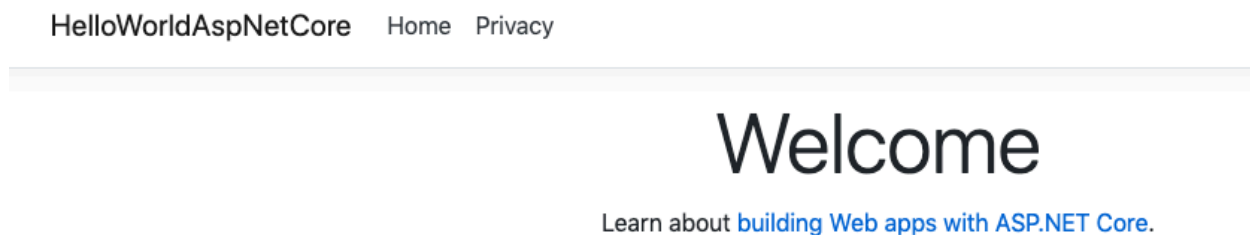
應用程式會開始監聽通訊埠 8080。

```
Hosting environment: Production
Content root path: /home/atameldev/HelloWorldAspNetCore
Now listening on: http://[::]:8080
Application started. Press Ctrl+C to shut down.
```

如要確認應用程式是否正在執行，請點按右上方的網頁預覽按鈕，然後選取「透過以下通訊埠預覽：8080」。



您會看到預設的 ASP.NET Core 網頁：



確認應用程式正在執行後，請按下 Ctrl+C 鍵關閉應用程式。

步驟 5 · 約 5 分鐘

5. 將 ASP.NET Core 應用程式封裝至 Docker 容器

接下來，請準備以容器形式執行的應用程式。第一步是定義容器及其內容。

在應用程式的根目錄中，建立 `Dockerfile` 來定義 Docker 映像檔。

```
touch Dockerfile
```

使用慣用的編輯器 (`vim`、`nano`、`emacs` 或 Cloud Shell 的程式碼編輯器)，將下列內容新增至 `Dockerfile`。

```
# Use Microsoft's official build .NET image.
# https://hub.docker.com/_/microsoft-dotnet-core-sdk/
FROM mcr.microsoft.com/dotnet/sdk:5.0-alpine AS build
WORKDIR /app

# Install production dependencies.
# Copy csproj and restore as distinct layers.
COPY *.csproj ./
RUN dotnet restore

# Copy local code to the container image.
COPY . ./
WORKDIR /app

# Build a release artifact.
RUN dotnet publish -c Release -o out

# Use Microsoft's official runtime .NET image.
# https://hub.docker.com/_/microsoft-dotnet-core-aspnet/
FROM mcr.microsoft.com/dotnet/aspnet:5.0-alpine AS runtime
WORKDIR /app
COPY --from=build /app/out ./

# Make sure the app binds to port 8080
ENV ASPNETCORE_URLS http://*:8080

# Run the web service on container startup.
ENTRYPOINT ["dotnet", "HelloWorldAspNetCore.dll"]
```

`Dockerfile` 中包含一項重要設定，也就是應用程式監聽傳入流量的通訊埠 (8080)。方法是設定 `ASPNETCORE_URLS` 環境變數，ASP.NET Core 應用程式會使用這項變數判斷要監聽哪個通訊埠。

儲存這項 `Dockerfile`。現在來建構映像檔：

```
docker build -t gcr.io/${GOOGLE_CLOUD_PROJECT}/hello-dotnet:v1 .
```

完成後 (下載及解壓縮所有內容需要一些時間)，您會看到映像檔已建構並儲存在本機：

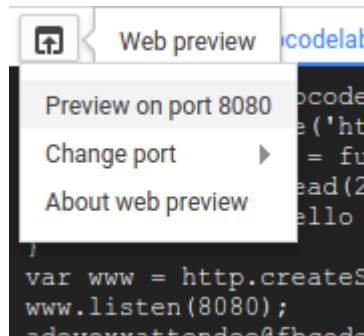
```
docker images
```

REPOSITORY	TAG
gcr.io/yourproject-XXXX/hello-dotnet	v1

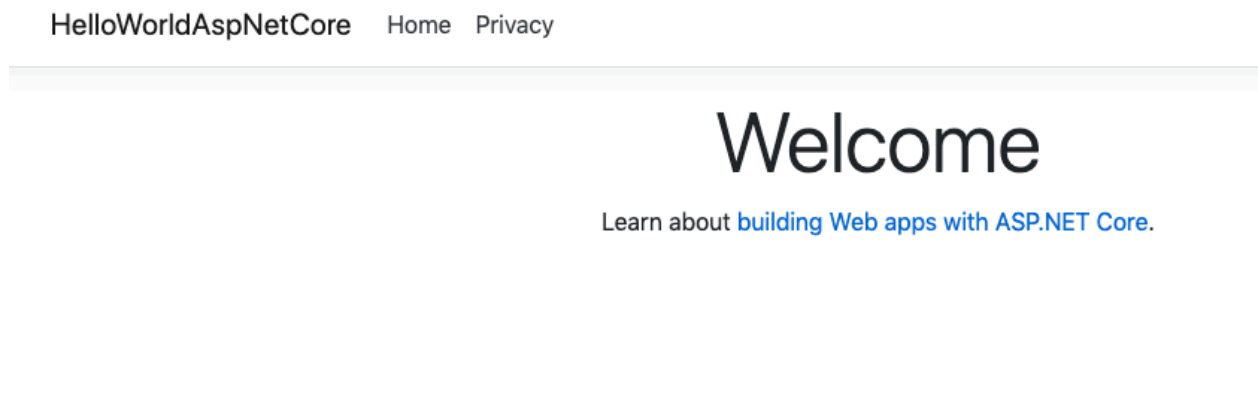
使用下列指令在本機測試映像檔，這會從您新建立的容器映像檔，在本機的 8080 埠執行 Docker 容器：

```
docker run -p 8080:8080 gcr.io/${GOOGLE_CLOUD_PROJECT}/hello-dotnet:v1
```

再次善用 Cloud Shell 的網頁預覽功能：



新分頁中應會顯示預設的 ASP.NET Core 網頁。

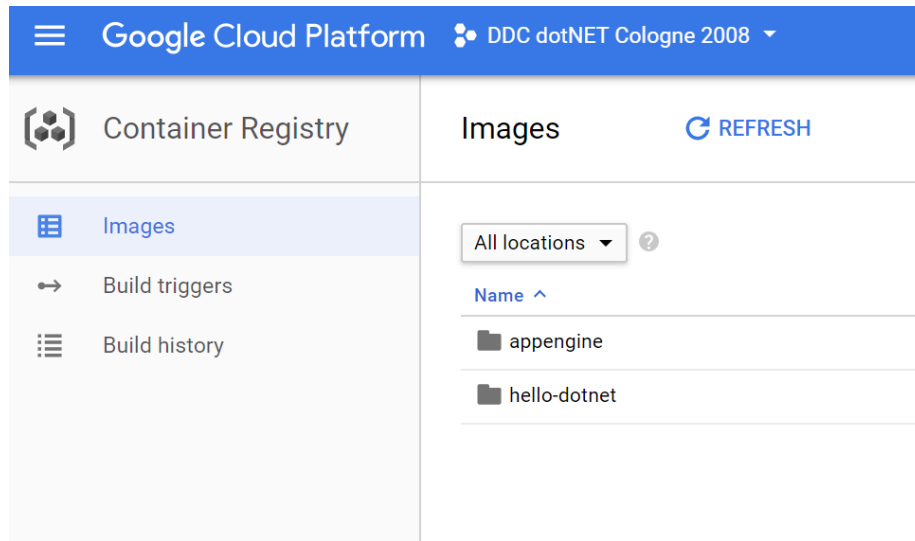


確認應用程式在本機 Docker 容器中正常運作後，即可按下 `Ctrl-C` 停止執行中的容器。

現在映像檔已如預期運作，您可以將其推送至 [Google Container Registry](#)，這是 Docker 映像檔的私人存放區，可從每個 Google Cloud 專案存取 (但也可從 Google Cloud Platform 外部存取)：

```
docker push gcr.io/${GOOGLE_CLOUD_PROJECT}/hello-dotnet:v1
```

如果一切順利，過一會兒後，您應該就能在「Container Registry」部分看到容器映像檔。此時，您已擁有專案適用的 Docker 映像檔，Kubernetes 幾分鐘後就能存取及協調這個映像檔。



請注意，雖然我們在此使用了登錄檔的通用網域 (`gcr.io`)，但您也可以更明確地指定要使用的可用區和值區。詳情請參閱：https://cloud.google.com/container-registry/docs/#pushing_to_the_registry

如要查看容器映像檔在 Google Cloud Storage 中的儲存位置，請點選這個連結：
<https://console.cloud.google.com/storage/browser/> (完整連結應為
https://console.cloud.google.com/project/PROJECT_ID/storage/browser/)。

步驟 6 · 約 10 分鐘

6. 使用 Istio 建立 Kubernetes/GKE 叢集

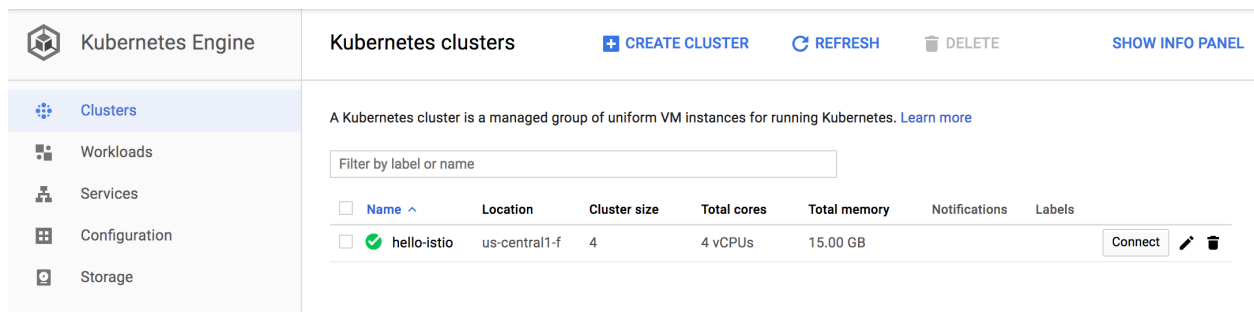
首先，請確認已啟用 Kubernetes Engine API：

```
gcloud services enable container.googleapis.com
```

建立 Kubernetes 叢集。如要變更地區，請按照下列步驟操作：

```
gcloud container clusters create hello-istio \
  --cluster-version=latest \
  --machine-type=n1-standard-2 \
  --num-nodes=4 \
  --region europe-west1
```

請稍候片刻，等待系統為您設定叢集。這項資訊會顯示在 Google Cloud Platform 控制台的 Kubernetes Engine 專區。



Name	Location	Cluster size	Total cores	Total memory	Notifications	Labels
hello-istio	us-central1-f	4	4 vCPUs	15.00 GB		

在本程式碼研究室中，我們將從 istio.io 下載並安裝 Istio。您也可以選擇其他安裝方式，包括 [GKE 的 Istio 外掛程式](#) 和 [Anthos 服務網格](#)。後續的應用程式步驟適用於任何 Istio 安裝作業。

首先，請下載 Istio 用戶端和範例。[Istio 版本頁面](#) 提供多個作業系統的下載構件。以我們的案例來說，可以使用便利的指令，下載並解壓縮目前平台的最新版本：

```
curl -L https://istio.io/downloadIstio | sh -
```

指令碼會顯示已下載的 Istio 版本：

```
Istio has been successfully downloaded into the istio-1.8.1 folder on your system.
```

安裝目錄包含範例應用程式和 `istioctl` 用戶端二進位檔。切換至該目錄：

```
cd istio-1.8.1
```

複製並貼上提供的指令，將 `bin` 目錄新增至 `PATH`，以便使用 `istioctl`：

```
export PATH="$PATH:/home/<YOURHOMEID>/istio-1.8.1/bin"
```

確認 `istioctl` 是否可用，方法是檢查叢集是否已準備好使用 Istio：

```
istioctl x precheck
```

畫面上應會顯示「Install Pre-Check passed! The cluster is ready for Istio installation.」訊息。

使用示範設定檔安裝 Istio：

```
istioctl install --set profile=demo
```

Istio 現已安裝在叢集中。

自動注入 Sidecar

如要開始使用 Istio，您不需要對應用程式進行任何變更。設定及執行服務時，系統會自動將 Envoy Sidecar 插入服務的每個 Pod。

如要啟用這項功能，您必須為微服務使用的命名空間（「default」）啟用 Sidecar 插入功能。方法是套用標籤：

```
kubectl label namespace default istio-injection=enabled
```

如要確認標籤是否已順利套用，請執行下列指令：

```
kubectl get namespace -L istio-injection
```

輸出內容會確認已為預設命名空間啟用補充資訊植入功能：

NAME	STATUS	AGE	ISTIO-INJECTION
default	Active	3m	enabled
istio-system	Active	63s	disabled
...			

7. 驗證安裝

Istio 隨附三項服務：`istiod` 控制層，以及分別命名為 `istio-ingressgateway` 和 `istio-egressgateway` 的 Ingress 和 Egress 閘道 (可視為「網際網路其餘部分的 Sidecar Proxy」)。

```
kubectl get svc -n istio-system
```

輸出內容應如下所示：

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
AGE			
istio-egressgateway	ClusterIP	10.55.252.182	<none>
istio-ingressgateway	LoadBalancer	10.55.250.185	35.233.118.42
istiod	ClusterIP	10.55.253.217	<none>

Ingress Gateway 的類型為 `LoadBalancer`，因此可從網際網路存取；其他閘道只需要可從叢集內存取即可。

接著，請確認已部署對應的 Kubernetes Pod，且所有容器都已啟動並執行：

```
kubectl get pods -n istio-system
```

所有 Pod 執行完畢後，即可繼續操作。

NAME	READY	STATUS
istio-egressgateway-674988f895-m6tk4	1/1	Running
istio-ingressgateway-6996f7dcc8-7lvm2	1/1	Running
istiod-6bf5fc8b64-j79hj	1/1	Running

- `istiod`：Istio 控制層。處理 Proxy 補充容器的設定和程式設計、服務探索、憑證發布和補充容器插入作業
 - `ingress gateway`：處理來自叢集外部的要求。
 - `egress gateway`：處理叢集外部端點的外送要求。
-

步驟 8 · 約 3 分鐘

8. 部署應用程式

確認 Istio 已安裝並執行後，即可部署 ASP.NET Core 應用程式。

部署與服務

首先，請使用慣用的編輯器 (`vim`, `nano`, `emacs` 或 Cloud Shell 的程式碼編輯器) 建立 `aspnetcore.yaml` 檔案，並為應用程式定義 Kubernetes Deployment 和 Service：

請務必將 `YOUR-PROJECT-ID` 替換為專案 ID

```

apiVersion: v1
kind: Service
metadata:
  name: aspnetcore-service
  labels:
    app: aspnetcore
spec:
  ports:
    - port: 8080
      name: http
  selector:
    app: aspnetcore
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: aspnetcore-v1
spec:
  replicas: 1
  selector:
    matchLabels:
      app: aspnetcore
      version: v1
  template:
    metadata:
      labels:
        app: aspnetcore
        version: v1
    spec:
      containers:
        - name: aspnetcore
          image: gcr.io/YOUR-PROJECT-ID/hello-dotnet:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080

```

檔案內容是部署應用程式的標準部署和服務，不含任何 Istio 專屬項目。

使用 `kubectl` 將服務部署至預設命名空間：

```
kubectl apply -f aspnetcore.yaml
```

```

service "aspnetcore-service" created
deployment.extensions "aspnetcore-v1" created

```

確認 Pod 正在執行：

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
aspnetcore-v1-6cf64748-mddb	2/2	Running	0	34s

您是否注意到每個 Pod 都包含兩個容器？包括應用程式容器和 Istio Proxy 補充容器。Pod 啟動時，系統會自動插入 Proxy。

Gateway 和 VirtualService

如要允許輸入流量抵達網格，您需要建立 [Gateway](#) 和 [VirtualService](#)。

閘道會為 HTTP/TCP 流量設定負載平衡器，通常在網格邊緣運作，以便為應用程式啟用 Ingress 流量。VirtualService 會定義規則，控管 Istio 服務網格中服務的要求轉送方式。

建立 `aspnetcore-gateway.yaml` 檔案來定義 Gateway：

```
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: aspnetcore-gateway
spec:
  selector:
    istio: ingressgateway # use istio default controller
  servers:
    - port:
        number: 80
        name: http
        protocol: HTTP
      hosts:
        - "*"

```

建立 `aspnetcore-virtualservice.yaml` 檔案來定義 VirtualService：

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: aspnetcore-virtualservice
spec:
  hosts:
  - "*"
  gateways:
  - aspnetcore-gateway
  http:
  - route:
    - destination:
        host: aspnetcore-service
```

執行 kubectl 指令，使用下列指令部署 Gateway：

```
kubectl apply -f aspnetcore-gateway.yaml
```

這項指令會產生下列輸出：

```
gateway.networking.istio.io "aspnetcore-gateway" created
```

接著，執行下列指令來部署 VirtualService：

```
kubectl apply -f aspnetcore-virtualservice.yaml
```

這項指令會產生下列輸出：

```
virtualservice.networking.istio.io "aspnetcore-virtualservice" created
```

確認一切運作正常：

```
kubectl get gateway
```

NAME	AGE
aspnetcore-gateway	28s

```
kubectl get virtualservice
```

NAME	AGE
aspnetcore-virtualservice	33s

恭喜！您已部署支援 Istio 的應用程式。接著，您會看到正在使用的應用程式。

步驟 9 · 約 2 分鐘

9. 測試應用程式

您終於可以查看應用程式的實際運作情形。您需要取得閘道的外部 IP 和通訊埠。該項目會列在「EXTERNAL-IP」下方：

```
kubectl get svc istio-ingressgateway -n istio-system
```

將外部 IP 和通訊埠匯出至 `GATEWAY_URL` 變數：

```
export INGRESS_HOST=$(kubectl -n istio-system get service istio-ingressgateway -o  
jsonpath='{.status.loadBalancer.ingress[0].ip}')  
  
export INGRESS_PORT=$(kubectl -n istio-system get service istio-ingressgateway -o  
jsonpath='{.spec.ports[?(@.name=="http2")].port}')  
  
export GATEWAY_URL=$INGRESS_HOST:$INGRESS_PORT
```

使用 `curl` 測試應用程式。服務應會傳回 200 回應代碼：

```
curl -o /dev/null -s -w "%{http_code}\n" http://${GATEWAY_URL}/
```

或者，你也可以開啟瀏覽器，前往 `http://<gatewayurl>` 查看應用程式：

HelloWorldAspNetCore Home Privacy

Welcome

Learn about [building Web apps with ASP.NET Core](#).

10. 恭喜！

您已將簡單的 ASP.NET Core 應用程式部署至 [Google Kubernetes Engine](#) (GKE) 上執行的 Kubernetes，並設定由 Istio 管理。

您可能會想「Istio 有什麼好處？」。這個問題非常好。目前讓 Istio 管理這個應用程式沒有任何優勢。在實驗室的第二部分，我們將進一步探索 Istio 的功能，例如指標、追蹤、動態流量管理、服務視覺化和容錯植入。

後續步驟

- [使用 Istio 將 ASP.NET Core 應用程式部署至 GKE \(第 2 部分\)](#)。
- 進一步瞭解 [Istio](#)。
- 進一步瞭解 [Kubernetes](#)。
- 進一步瞭解 [Google Kubernetes Engine](#)。
- 進一步瞭解 [Google Cloud Platform 上的 .NET](#)。

授權

這項內容採用的授權為 Creative Commons 姓名標示 2.0 通用授權。

11. 清除所用資源

如果不想繼續進行實驗室的第二部分，可以刪除應用程式並解除安裝 Istio，或直接刪除 Kubernetes 叢集。

刪除應用程式

如要刪除該應用程式：

```
kubectl delete -f aspnetcore-gateway.yaml
Kubectl delete -f aspnetcore-virtualservice.yaml
kubectl delete -f aspnetcore.yaml
```

如要確認應用程式已移除，請按照下列步驟操作：

```
kubectl get gateway
kubectl get virtualservices
kubectl get pods
```

解除安裝 Istio

如要刪除 Istio，請執行下列操作：

```
kubectl delete -f install/kubernetes/istio-demo-auth.yaml
```

確認 Istio 已移除：

```
kubectl get pods -n istio-system
```

刪除 Kubernetes 叢集

```
gcloud container clusters delete hello-istio
```
